

Tarea 9 — Simulación de base de datos distribuida y fragmentación

Jhoan Camilo Arango Ortiz · 2º ASIR online · Administración de Sistemas Gestores de Bases de Datos

1. Introducción

En esta tarea he simulado cómo funciona una base de datos distribuida usando MySQL. La idea es que en un sistema real los datos no están todos en el mismo servidor, sino repartidos entre varios nodos. Aquí lo simulo creando distintas bases de datos dentro del mismo MySQL que representan cada uno de esos nodos.

Hay dos formas de fragmentar. La fragmentación horizontal divide las filas de una tabla entre nodos, por ejemplo poniendo los empleados de Madrid en un sitio y los de Barcelona en otro. La fragmentación vertical separa las columnas de una misma entidad en tablas distintas, enlazadas por el mismo ID.

2. Fragmentación horizontal

He creado las bases de datos **empresa_madrid** y **empresa_barcelona**, que simulan los nodos de cada sede. Las dos tienen la misma tabla **empleados** con los mismos campos, pero con datos distintos: cada una guarda solo los empleados que le corresponden. Los IDs van del 1 al 3 en Madrid y del 4 al 6 en Barcelona para que no haya solapamiento.

Nodo Madrid

```

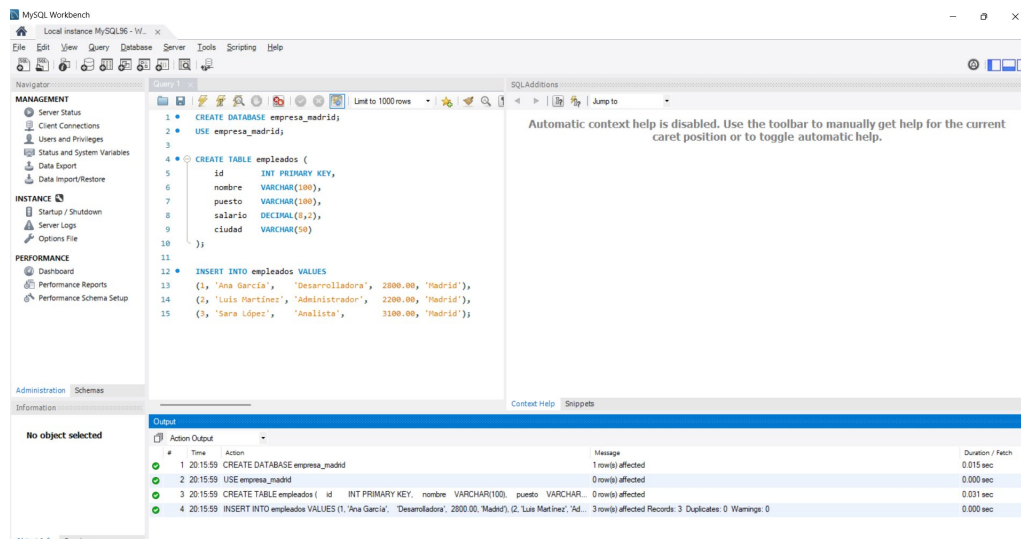
CREATE DATABASE empresa_madrid;

USE empresa_madrid;

CREATE TABLE empleados (
    id          INT PRIMARY KEY,
    nombre      VARCHAR(100),
    puesto      VARCHAR(100),
    salario     DECIMAL(8,2),
    ciudad      VARCHAR(50)
);

INSERT INTO empleados VALUES
(1, 'Ana García',    'Desarrolladora', 2800.00, 'Madrid'),
(2, 'Luis Martínez', 'Administrador',  2200.00, 'Madrid'),
(3, 'Sara López',    'Analista',      3100.00, 'Madrid');

```



Ejecución en MySQL Workbench: empresa_madrid creada con sus 3 registros.

Nodo Barcelona

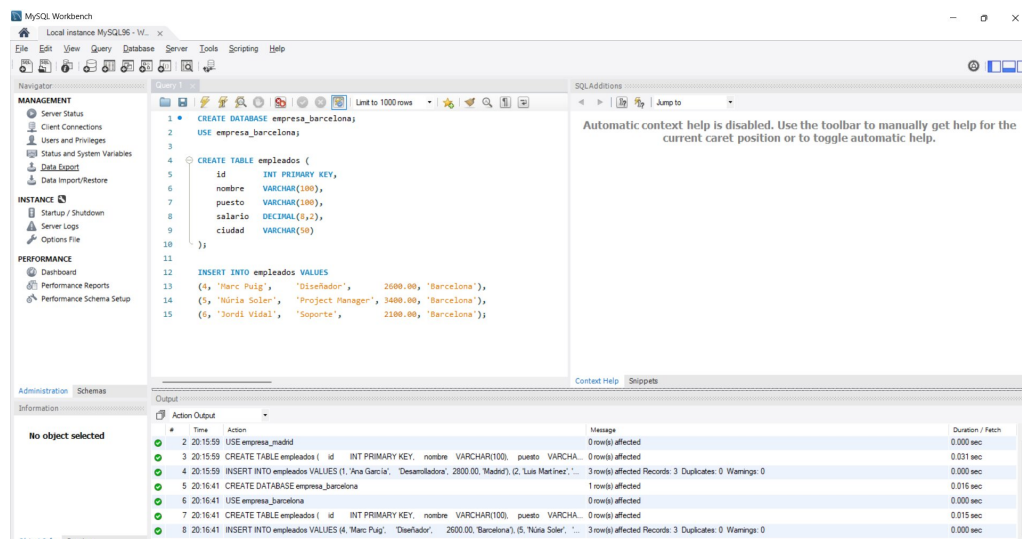
```

CREATE DATABASE empresa_barcelona;
USE empresa_barcelona;

CREATE TABLE empleados (
    id          INT PRIMARY KEY,
    nombre      VARCHAR(100),
    puesto      VARCHAR(100),
    salario     DECIMAL(8,2),
    ciudad      VARCHAR(50)
);

INSERT INTO empleados VALUES
(4, 'Marc Puig',      'Diseñador',      2600.00, 'Barcelona'),
(5, 'Núria Soler',    'Project Manager', 3400.00, 'Barcelona'),
(6, 'Jordi Vidal',    'Soporte',        2100.00, 'Barcelona');

```



Ejecución en MySQL Workbench: empresa_barcelona creada con sus 3 registros.

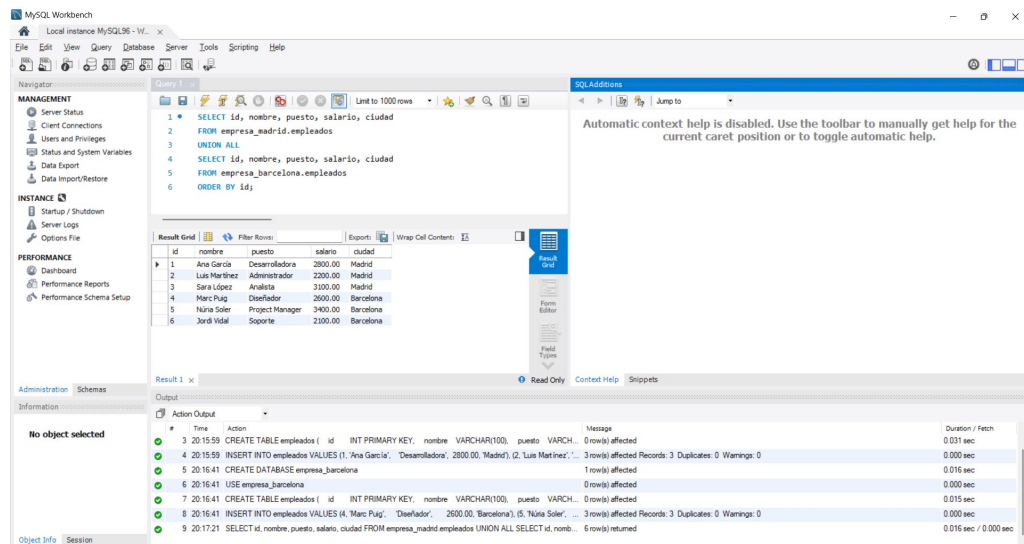
3. Reconstrucción horizontal con UNION ALL

Para ver todos los empleados juntos uso **UNION ALL**, que apila los resultados de las dos consultas. He usado UNION ALL en vez de UNION porque no hay duplicados entre las dos bases de datos, así que no tiene sentido que MySQL haga la comprobación extra. El ORDER BY al final ordena todo por ID.

```

SELECT id, nombre, puesto, salario, ciudad
FROM empresa_madrid.empleados
UNION ALL
SELECT id, nombre, puesto, salario, ciudad
FROM empresa_barcelona.empleados
ORDER BY id;

```



Resultado de UNION ALL: los 6 empleados de ambos nodos en una sola consulta.

4. Fragmentación vertical

He creado la base de datos **empresa_vertical** con dos tablas: **empleados_personal** guarda el nombre, email y teléfono, y **empleados_salario** guarda el puesto, el salario y el departamento. Las dos comparten el campo **id** para poder cruzarlas con un JOIN.

Tiene sentido separarlo así cuando hay datos que no todo el mundo debería ver. Por ejemplo, alguien de soporte puede necesitar el teléfono de un compañero sin tener acceso a lo que cobra.

```

CREATE DATABASE empresa_vertical;
USE empresa_vertical;

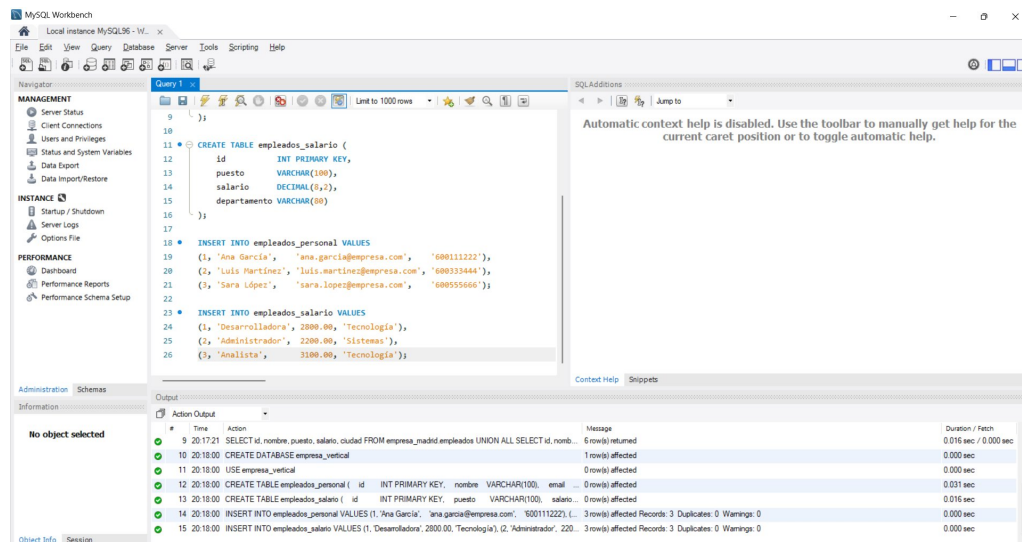
CREATE TABLE empleados_personal (
    id          INT PRIMARY KEY,
    nombre      VARCHAR(100),
    email       VARCHAR(150),
    telefono    VARCHAR(20)
);

CREATE TABLE empleados_salario (
    id          INT PRIMARY KEY,
    puesto      VARCHAR(100),
    salario     DECIMAL(8,2),
    departamento VARCHAR(80)
);

INSERT INTO empleados_personal VALUES
(1, 'Ana García', 'ana.garcia@empresa.com', '600111222'),
(2, 'Luis Martínez', 'luis.martinez@empresa.com', '600333444'),
(3, 'Sara López', 'sara.lopez@empresa.com', '600555666');

INSERT INTO empleados_salario VALUES
(1, 'Desarrolladora', 2800.00, 'Tecnología'),
(2, 'Administrador', 2200.00, 'Sistemas'),
(3, 'Analista', 3100.00, 'Tecnología');

```



Creación de empresa_vertical con las dos tablas fragmentadas y los datos insertados.

5. Reconstrucción vertical con JOIN

Con un **JOIN** sobre el campo **id** reconstruyo la ficha completa de cada empleado. El resultado muestra todos los campos de las dos tablas en una sola fila, como si nunca se hubieran separado.

```
SELECT
    p.id,
    p.nombre,
    p.email,
    p.telefono,
    s.puesto,
    s.salario,
    s.departamento
FROM empleados_personal p
JOIN empleados_salario s ON p.id = s.id
ORDER BY p.id;
```

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
SELECT
    p.telefono,
    s.puesto,
    s.salario,
    s.departamento
FROM empleados_personal p
JOIN empleados_salario s ON p.id = s.id
ORDER BY p.id;
```

The 'Result Grid' shows the output of the query, which consists of three rows of employee data:

id	nombre	email	telefono	puesto	salario	departamento
1	Ana Garcia	ana.garcia@empresa.com	600111222	Desarrolladora	2800.00	Tecnologia
2	Luis Martinez	luis.martinez@empresa.com	600333444	Administrador	2200.00	Sistemas
3	Sara Lopez	sara.lopez@empresa.com	600555666	Analista	3100.00	Tecnologia

The 'Output' tab at the bottom shows the execution log, including the creation of the database, tables, and the execution of the JOIN query.

Resultado del JOIN: ficha completa de cada empleado reconstruida desde las dos tablas.

6. Preguntas teóricas

¿Qué ventajas tiene la fragmentación?

Lo más evidente es el rendimiento. Al repartir los datos, cada nodo maneja menos registros y las consultas que solo afectan a un nodo van más rápido porque no tienen que leer datos que no necesitan. También mejora la disponibilidad: si cae el servidor de Barcelona, los datos de Madrid siguen funcionando sin problema.

La fragmentación vertical además permite controlar el acceso a columnas sensibles sin montar vistas ni estructuras más complicadas. Cada tabla puede tener sus propios permisos.

¿Qué problemas puede generar?

El principal problema es que las consultas se complican bastante. Cualquier consulta que necesite datos de varios nodos requiere un UNION o un JOIN entre bases de datos distintas, que es más lento que consultar una sola tabla local.

Mantener la coherencia también es complicado: si actualizas un dato en un nodo y te olvidas del otro, tienes inconsistencias que no siempre son fáciles de detectar. A nivel de administración se añade trabajo: más servidores que vigilar, backups que coordinar y cambios de esquema que hay que aplicar en todos los nodos a la vez. En resumen, fragmentar tiene sentido cuando la escala lo justifica, pero para sistemas pequeños probablemente complica más de lo que ayuda.